

Data Analysis Project Report

Peter Francek

Abstract

Learning optimal assignments between structured objects is central to tasks such as point cloud matching and graph similarity. Exact combinatorial solvers provide ground-truth costs but are non-differentiable and computationally expensive.

We compare two neural architectures for approximating assignment-based distances: a Sinkhorn-based model that learns a differentiable transport plan and a permutation-invariant DeepSets model that captures global structural statistics. We evaluate both approaches on synthetic point cloud matching tasks and on molecular graph edit distance (GED) prediction using the SAMPL dataset under different cost settings and present their strengths and weaknesses.

1 Introduction

In this report, we study the problem of learning optimal assignments between structured objects represented as multisets.

While exact matching algorithms such as the Hungarian method provide optimal solutions, they are often computationally expensive and non-differentiable, limiting their direct integration into learning systems. Differentiable optimal transport, particularly through the Sinkhorn algorithm, offers a principled way to approximate discrete matchings with continuous relaxations. At the same time, permutation-invariant neural architectures such as DeepSets might provide a flexible alternative that avoids explicit matching altogether by learning global set representations.

We investigate how these two paradigms compare when learning similarity functions between structured objects and whether architectures like DeepSets can match or surpass Sinkhorn-based approaches. We study both controlled synthetic point cloud datasets, where ground-truth distances are precisely defined, and a real-world molecular

graph dataset derived from the SAMPL challenges, where similarity is defined via approximate GED. Our goal is to understand when explicit matching through Sinkhorn-based optimal transport is beneficial, and whether simpler permutation-invariant aggregation suffices.

We implement and systematically compare DeepSets and Sinkhorn-based architectures under multiple data regimes and cost definitions, and we analyze the impact of dimensionality, set cardinality, and cost structure on performance and convergence behavior. Through extensive experiments, we highlight the trade-offs between inductive bias, computational efficiency, and performance in learning similarity functions.

2 Background

This section reviews the theoretical background required for our study, including the formal problem definition, optimal transport with Sinkhorn regularization, and permutation-invariant neural architectures.

2.1 Problem Definition

Let $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_m\}$ denote two unordered sets of objects, where each element $x \in A \cup B$ is represented by a feature vector in $\mathbb{R}^d$. These elements may correspond to points in a geometric space or nodes in a graph with associated attributes. The objective is to predict a scalar similarity or cost value $y \in \mathbb{R}$ that reflects how well the points in A can be matched to those in B .

In the supervised setting considered here, the training data consists of tuples (A, B, y) , where the ground-truth target y is computed from an optimal assignment problem. Let

$$c : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$$

be a pairwise cost function measuring the cost of matching two elements (Euclidean distance or a

different metric). We define the cost matrix $C \in \mathbb{R}^{n \times m}$ by

$$C_{ij} = c(a_i, b_j).$$

In the standard case where $|A| = |B| = n$, the ground-truth similarity y is given by the solution to the Linear Sum Assignment Problem (LSAP):

$$\min_{P \in \mathcal{P}_n} \langle C, P \rangle_F,$$

where $\mathcal{P}_n$ denotes the set of permutation matrices of size $n \times n$, and

$$\langle C, P \rangle_F = \sum_{i,j} C_{ij} P_{ij}$$

is the Frobenius inner product. The learning challenge is thus to approximate this combinatorial minimum using a differentiable architecture. The model must be invariant to permutations of the elements within each set and robust to varying set sizes.

2.2 Graph Edit Distance

Graph Edit Distance (GED) is a fundamental measure of similarity between two graphs, defined as the minimum cost of a sequence of edit operations required to transform one graph G_1 into another G_2 . The standard edit operations include:

- **Insertion:** Adding a new node or edge.
- **Deletion:** Removing an existing node or edge.
- **Substitution:** Changing the label or attributes of a node or edge.

Formally, if $\mathcal{P}(G_1, G_2)$ is the set of all valid edit paths transforming G_1 to G_2 , and $c(e)$ is the cost of an operation e , the GED is given by:

$$\text{GED}(G_1, G_2) = \min_{(e_1, \dots, e_k) \in \mathcal{P}(G_1, G_2)} \sum_{i=1}^k c(e_i) \quad (1)$$

Since computing the exact GED is NP-hard, the ground-truth labels are usually computed using a bipartite graph matching heuristic (Riesen and Bunke, 2009). That is, for every pair of graphs (G_1, G_2) we construct a cost matrix that accounts for:

- **Node/Edge substitution:** A cost incurred when matching two nodes/edges with differing labels.

- **Node insertion/deletion:** Fixed costs for inserting or deleting nodes. Deleting a node implicitly removes all its incident edges, whose costs are accounted for separately.
- **Edge insertion/deletion:** Penalties associated with adding or removing edges, typically determined by differences in node degrees.

The target GED is then approximated by solving the LSAP on such a cost matrix.

2.3 Sinkhorn Algorithm

The Sinkhorn algorithm is an iterative method used to solve the entropy-regularized Optimal Transport (OT) problem (Cuturi, 2013). In the context of matching two sets, such as nodes in two graphs, we are given a cost matrix $C \in \mathbb{R}^{n \times m}$ and seek a transport plan P that minimizes the total matching cost.

By adding an entropy regularization term, the optimal solution P is guaranteed to take the following form:

$$P = \text{diag}(u) K \text{diag}(v) \quad (2)$$

where $K = \exp(-C/\epsilon)$ is a kernel matrix calculated element-wise from the cost matrix, and $\epsilon > 0$ is a regularization parameter. The vectors $u \in \mathbb{R}^n$ and $v \in \mathbb{R}^m$ are scaling factors that ensure the rows and columns of P sum to the required marginal distributions a and b (typically vectors of ones for a valid assignment). The Sinkhorn algorithm finds these vectors by iteratively updating them:

$$\begin{aligned} u^{(k+1)} &= a \oslash (K v^{(k)}) \\ v^{(k+1)} &= b \oslash (K^\top u^{(k+1)}) \end{aligned}$$

where $\oslash$ denotes element-wise division. This process continues until P converges, providing a "soft" assignment between the two sets (Peyré and Cuturi, 2020).

2.4 DeepSets Models

DeepSets are neural architectures designed to operate on unordered sets while guaranteeing permutation invariance. A DeepSets model typically consists of a per-element encoder ψ followed by a symmetric aggregation operator (such as sum), and a final predictor ϕ .

Formally, given a set $X = \{x_1, \dots, x_n\}$, the model computes a set-level embedding z_X :

$$z_X = \phi \left(\sum_{x \in X} \psi(x) \right) \quad (3)$$

where ψ maps each element to a latent representation, and ϕ is a neural network that transforms the pooled vector into the final set representation (Zaheer et al., 2018).

3 Methodology

3.1 Datasets

To evaluate the proposed methods, we utilize two distinct datasets: a controlled synthetic dataset of point clouds and a real-world dataset of molecular graphs from the SAMPL challenges (SAMPL challenges, 2024).

3.1.1 Synthetic Point Cloud Data

We generate a synthetic dataset consisting of M pairs of unordered point sets (A, B) , where each point lies in $\mathbb{R}^d$. This dataset allows us to verify model performance in a controlled environment with known ground-truth matching costs.

Generation Process: For each pair, set A is sampled uniformly in the unit hypercube $[0, 1]^d$. Set B is generated by perturbing the points in A to efficiently span a full range of similarities. Specifically, for each point $a_i \in A$, a corresponding point b_i is created by displacing a_i in a random direction by a distance derived from a target similarity score $s_i \sim \mathcal{N}(s_{\text{target}}, \sigma)$. The displacement distance is given by $d_i = (1 - s_i)$.

To ensure the models are robust to varying input sizes, the generation process allows for variable set cardinalities. For any given pair, the sizes $|A|$ and $|B|$ can be sampled independently from a predefined range.

Ground Truth Metrics: For the synthetic data, we define the ground-truth similarity using one of two distinct distance metrics, allowing us to evaluate the model’s flexibility:

- **Euclidean Distance:** The standard cost is computed using the Hungarian algorithm (Linear Sum Assignment) on the pairwise Euclidean distance matrix. This finds the bijective matching between points that minimizes the sum of squared distances.
- **Hausdorff Distance:** Alternatively, we generate labels based on the Hausdorff distance, which measures the "worst-case" mismatch between the sets. It is defined as:

$$H(A, B) = \max \left(\sup_{a \in A} \inf_{b \in B} \|a - b\|, \sup_{b \in B} \inf_{a \in A} \|a - b\| \right) \quad (4)$$

This metric represents the maximum distance from a point in one set to the nearest point in the other.

3.1.2 Molecular Graph Data (SAMPL)

To test the models on structured data, we use the SAMPL dataset, which consists of small organic molecules relevant to physical property prediction.

Graph Construction: The molecules, originally provided as SMILES strings, are converted into undirected graphs using RDKit. Atoms are represented as nodes and chemical bonds as edges. While the raw graphs contain rich chemical information (hybridization, aromaticity, bond type), for the purpose of this study, we extract just a tuple: (atomic_number, degree). We compute the ground truth labels using a bipartite graph matching approximation on every pair of graphs.

Preprocessing: For model input, the graphs are treated as multisets of node features. To handle varying molecule sizes in batches, the sets are padded with dummy nodes to the maximum graph size in the dataset, and binary masks are created to ensure the models ignore padding during computation. The target values (Y) and degree features are also standardized to zero mean and unit variance to stabilize training.

4 Model Architectures

We evaluate two distinct model architectures. **DeepSets** approach, and **Sinkhorn based model**, which explicitly models pairwise assignments via differentiable Optimal Transport. We compare these methods on both generated point cloud matching data and molecular graphs GED prediction.

4.1 DeepSets Architecture

The DeepSets architecture operates as a permutation-invariant feature extractor. It consists of two neural networks, ψ and ρ .

Given a set $X = \{x_1, \dots, x_N\}$, the model proceeds in three steps:

1. **Instance Embedding:** The shared encoder ψ is applied to each element x_i independently to generate local features.

$$h_i = \psi(x_i), \quad \forall x_i \in X \quad (5)$$

2. **Permutation-Invariant Aggregation:** The local features are aggregated to produce a global

set descriptor z_X .

$$z_X = \sum_i^N (h_i) \quad (6)$$

3. Set Processing: A second network ρ is used to produce the final set embedding.

$$\text{output}_X = \rho(z_X) \quad (7)$$

Both ψ and ρ are simple two-layer Multilayer Perceptrons (MLPs) with ReLU activations. To predict the distance between two sets X and Y , we generate their final embeddings output_X and output_Y and pass them to a regression head, which helps us learn a flexible distance function. Following common practice in Siamese and metric learning architectures, we construct a feature vector that captures both individual representations and their pairwise interactions (van Leeuwen et al., 2025), which we found helpful in learning similarity between the sets. Specifically, the combined feature vector

$$v_{\text{combined}} = [z_X, z_Y, |z_X - z_Y|, z_X \odot z_Y]$$

is processed by a simple two-layer MLP with a ReLU activation to regress the final scalar distance:

$$\hat{d}(X, Y) = \text{MLP}(v_{\text{combined}}) \quad (8)$$

This formulation enables the model to learn expressive, non-linear similarity functions.

4.2 Sinkhorn based models

The Sinkhorn-based models explicitly approximate the optimal matching between sets. To ensure a fair comparison, both variants use the same encoder ϕ_θ to embed the objects, which is architecturally equivalent to the DeepSets feature encoder (a two-layer MLP), but differ significantly in their cost matrix construction and normalization to account for different datasets.

4.2.1 Standard Sinkhorn (Point Clouds)

For point cloud matching, we employ the standard Sinkhorn formulation, where the cost matrix C is computed as pairwise cosine distance between the embedded points. And the transport matrix P is computed via standard Sinkhorn iterations. The final cost is obtained by summing the element-wise product between the transport matrix P and the cost matrix C :

$$d(X, Y) = \sum_{i,j} P_{ij} C_{ij} \quad (9)$$

4.2.2 Sinkhorn-GED (Molecular Graphs)

For the GED task, we implement a specific augmented cost matrix formulation adapted from recent literature on approximate GED (Wallnig et al., 2025) (Leonardi et al., 2025). Given two graphs with node counts N_1 and N_2 , we construct a square cost matrix $C \in \mathbb{R}^{(N_1+N_2) \times (N_1+N_2)}$ structured to allow substitutions, deletions, and insertions:

$$C = \begin{pmatrix} C_{\text{sub}} & C_{\text{del}} \\ C_{\text{ins}} & 0 \end{pmatrix} \quad (10)$$

where:

- $C_{\text{sub}} \in \mathbb{R}^{N_1 \times N_2}$ contains the pairwise substitution costs. It is computed as the cosine distance between node embeddings.
- $C_{\text{del}} \in \mathbb{R}^{N_1 \times N_1}$ is a diagonal matrix where the diagonal elements $C_{i, N_2+i} = \lambda_{\text{del}}$ are learnable deletion parameters. The off-diagonal elements are set to a large penalty value to prohibit invalid matches.
- $C_{\text{ins}} \in \mathbb{R}^{N_2 \times N_2}$ is a diagonal matrix where the diagonal elements $C_{N_1+j, j} = \lambda_{\text{ins}}$ are learnable insertion parameters. Similar to deletion, off-diagonals are set to a large penalty.
- The bottom-right block is strictly zero, acting as a sink for dummy-to-dummy matches.

To stabilize the Sinkhorn iterations for this partial matching formulation, we adopt the normalization trick proposed in (Brun et al., 2022), where the last row and column of the vectors are forced to 1. Given the similarity matrix $S = \exp(-C/\epsilon)$, the iteration step k updates row scalings r and column scalings c as follows:

$$r^{(k)} = \frac{1}{S c^{(k-1)}}, \quad r_{[n+1]}^{(k)} \leftarrow 1.0 \quad (11)$$

$$c^{(k)} = \frac{1}{S^T r^{(k)}}, \quad c_{[m+1]}^{(k)} \leftarrow 1.0 \quad (12)$$

This ensures that the dummy nodes, represented by the expansion of the matrix, can absorb arbitrary mass, effectively modeling the variable number of insertions and deletions required to match graphs of different sizes.

4.3 Training Procedure

All models are trained using a supervised regression objective, minimizing the Mean Squared Error (MSE) between the predicted and the ground-truth values.

We employ different optimization strategies tailored to each architecture. **DeepSets** model and **Sinkhorn (Point Clouds)** are optimized using standard Adam with a fixed learning rate found via grid search, while **Sinkhorn-GED** is optimized using AdamW with weight decay to prevent overfitting. To ensure training stability, we also employ cosine annealing for the learning rate to further stabilize the training and achieve better convergence.

From both the point cloud and the molecular graph datasets, we sample $M = 10,000$ pairs and split into 80% Training and 20% Testing. A further 20% of the training set is reserved for validation during hyperparameter tuning. Models are trained for 100 epochs with a batch size of 128.

4.4 Hyperparameter Optimization

To ensure a fair comparison, we conducted a grid search to identify optimal hyperparameters for each architecture. The search space included learning rate, embedding dimension size, hidden dimension size, number of Sinkhorn iterations, and Sinkhorn regularization parameter. During the search, models were trained for a reduced duration of 20 epochs. The best-performing configuration was then re-trained on the full training set for 100 epochs to report final test metrics.

4.5 Evaluation

We report model performance using different metrics such as standard MSE, Root MSE, Mean Absolute Error (MAE) but also Relative Error defined as $\frac{1}{N} \sum \frac{|\hat{y}-y|}{y+1}$, which assesses performance relative to the scale of the GED and Pearson correlation coefficient to see how well the predictions follow the ground-truth values.

5 Results and Analysis

We evaluate the proposed architectures on both the synthetic point cloud dataset and the SAMPL molecular GED prediction dataset. We analyze performance across varying conditions, including set cardinality, varying set sizes, and embedding dimensionality.

5.1 Performance on the Generated Point Cloud Data

For this experiment, we report the metrics for the two models within the best found hyperparameters, and later we evaluate how the models perform with respect to the sensitivity of varying dimensions of input points and varying set sizes. Sets with fixed

size have all equal size of N , while with random size the amount can differ in each pair of sets from $1 - N$.

Table 1: Performance Comparison on Generated Point Clouds ($N = 20, D = 10$).

Model	MSE	RMSE	MAE	Corr	Rel. Err
Sinkhorn	0.0003	0.0158	0.0124	0.9983	0.0082
DeepSets	0.0002	0.0148	0.0115	0.9985	0.0076

Table 1 reports the performance comparison between Sinkhorn and DeepSets on generated point clouds ($N = 20, D = 10$) data, with $N = 20$, which stands for the number of points in each set, and $D = 10$ suggesting the dimension of each point. Although DeepSets slightly outperforms Sinkhorn across all metrics, both models exhibit very high correlation with the ground truth, indicating that they can successfully learn to approximate the underlying metric.

5.1.1 Impact of per-point dimensionality

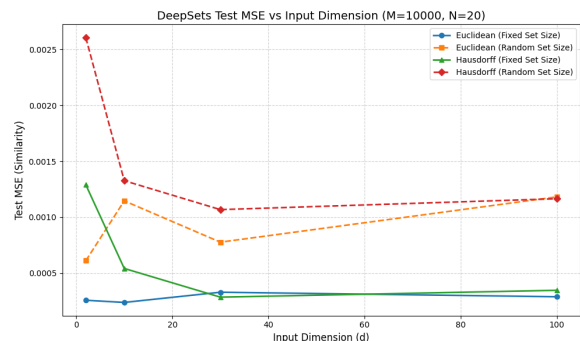


Figure 1: Comparison of MSE as the input dimension of points increases for the DeepSets model under fixed and random set size settings

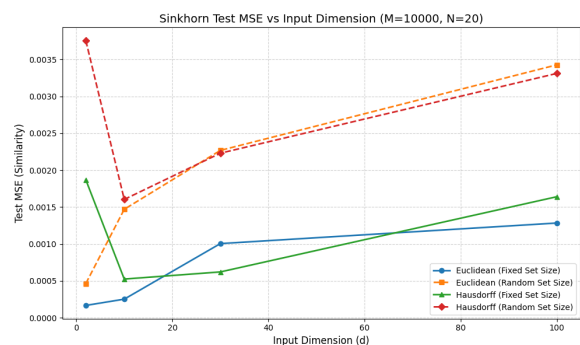


Figure 2: Comparison of MSE as the input dimension of points increases for the Sinkhorn model under fixed and random set size settings

Figures 1 and 2 compare the test MSE as a function of the input dimension of single points in sets for DeepSets and Sinkhorn models, respectively, under both fixed and random set size settings. For DeepSets, increasing the input dimension of points from just 2 to 10 substantially improves performance for the Hausdorff distance and then stabilizes. In contrast, Euclidean distance prediction remains relatively stable across dimensions. For the Sinkhorn model, however, the trend differs: while performance improves from very low dimension to moderate dimension for the Hausdorff distance, the test error increases again at higher dimensions, particularly from 10 and above. For the Euclidean distance, it seems the error seems to grow linearly with respect to increasing dimension size. Across both models, random set size consistently leads to higher error than fixed set size, indicating that variability in cardinality adds learning difficulty. Overall, DeepSets appears more stable with respect to dimensionality, whereas Sinkhorn exhibits greater sensitivity as the dimension increases, likely due to the curse of high dimensionality as the points in high dimensions become equally distant from each other. This error increase might also be reduced by varying different hyperparameters for such high dimensions.

5.1.2 Impact of Set Cardinality

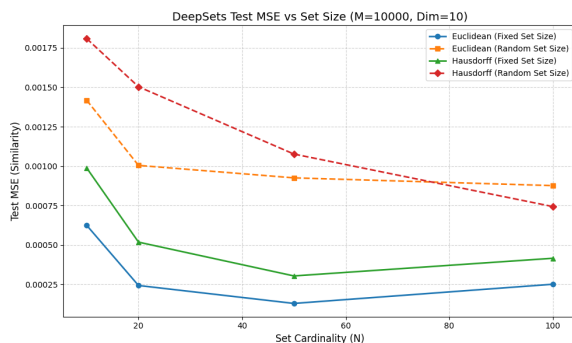


Figure 3: Comparison of MSE as the number of points in the set increases for the DeepSets model under fixed and random set size settings

We can see in Figures 3 and 4 that, contrary to the intuition that larger sets might be harder to match, we observe that the MSE generally decreases as the set cardinality N increases for both architectures. This trend holds across both Hausdorff and Euclidean metrics. This suggests that as the number of points grows, the set level statistics become more robust for learning the matching.

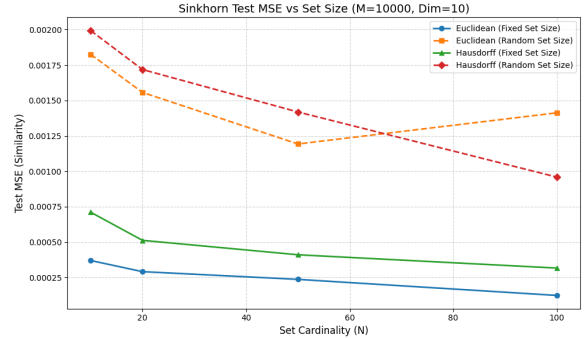


Figure 4: Comparison of MSE as the number of points in the set increases for the Sinkhorn model under fixed and random set size settings

In general, both models achieve comparable results, learning both Euclidean and Hausdorff distance-generated ground truth metrics. They seem to struggle where each pair of sets can have a different size, but with an increasing amount, they tend to learn better. In addition to that, Sinkhorn tends to worsen with higher dimensionality compared to the DeepSets model, the **DeepSets** offers a slight advantage also in training efficiency. By avoiding the iterative matrix scaling operations required by the Sinkhorn, DeepSets trains faster.

5.2 Performance on the SAMPL GED dataset

We evaluated the models on the SAMPL Molecular Graph dataset under two distinct cost configurations to test their adaptability to different metric definitions.

5.2.1 Simple Node Matching

We first consider a simplified GED setting in which only node edit operations are penalized, all node costs (insertion, substitution, deletion) are set to 1, and edge costs are set to 0. In this regime, the distance primarily reflects differences in node counts and substitutions, without accounting for structural edge discrepancies.

Model	MSE	RMSE	MAE	Corr	Rel. Err
Sinkhorn-GED	0.158	0.397	0.291	0.994	6.17%
DeepSets	0.327	0.572	0.434	0.993	7.44%

Table 2: Performance on Simplified GED (Node Ops=1.0, Edge Ops=0.0).

As shown in Table 2, the Sinkhorn-GED model outperforms the DeepSets model in all metrics, while both models achieve low relative error (6-7%).

This performance advantage can be attributed to the Sinkhorn architecture, since the problem reduces significantly to finding an optimal bipartite matching between atom sets. The Sinkhorn layer explicitly computes this transport plan (P), allowing the model to "count" exactly which nodes are matched and which are sent to the dummy bins (deletion/insertion cost). In contrast, DeepSets must infer these discrete matching statistics solely from aggregated global embeddings, which is a harder learning task.

5.2.2 Complex Weighted Costs

We next consider a weighted GED configuration with higher substitution penalties (node substitution = 6.0, insertion/deletion = 3.0) and equally for edge edit costs. In this setting, both node and edge discrepancies contribute substantially to the distance.

Model	MSE	RMSE	MAE	Corr	Rel. Err
Sinkhorn-GED	27.97	5.28	4.05	0.989	10.37%
DeepSets	8.55	2.92	2.07	0.997	5.53%

Table 3: Performance on Weighted GED (Sub=6.0, Ins/Del=3.0, Edge Ops included).

As shown in Table 3, DeepSets significantly outperforms Sinkhorn-GED across all evaluation metrics, reducing the MSE from 27.97 to 8.55 and nearly halving the relative error.

This reversal indicates that the GED depends less on precise node to node correspondences and more on overall structural differences between graphs. In this regime, permutation invariant aggregation appears to be a more effective structural representation. DeepSets can capture global discrepancies in node and edge statistics without explicitly solving a complex matching problem, which may explain its improved performance under weighted costs.

5.2.3 Convergence behavior

Figures 5 and 6 compare the training and validation loss curves (computed on standardized targets) for DeepSets and Sinkhorn-GED.

DeepSets exhibits a gradual decrease, stabilizing after approximately 30–40 epochs. The close alignment between the training and validation curves indicates stable optimization with no significant overfitting.

In contrast, Sinkhorn-GED converges much more rapidly. As shown in Figure 6, the model reaches a standardized loss of approx. 0.02 within

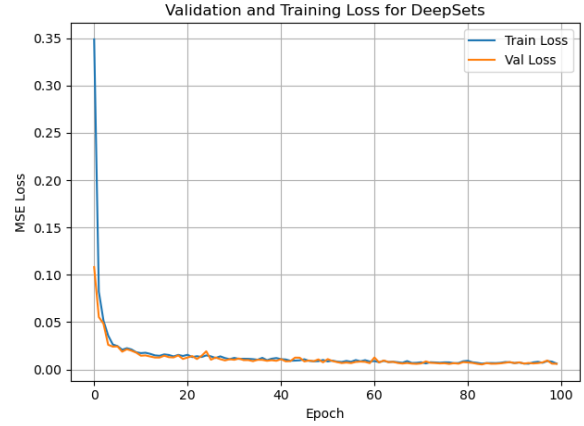


Figure 5: Validation and Training loss for the DeepSets in 100 epochs

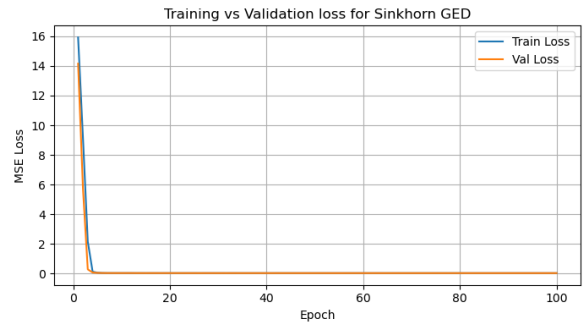


Figure 6: Validation and Training loss for the Sinkhorn-GED in 100 epochs

the first 10 epochs and stabilizes almost immediately. While this indicates highly efficient optimization, the lack of further improvement suggests the model quickly fits the dominant alignment structure but struggles to refine the remaining residual error.

Overall, Sinkhorn converges faster, whereas DeepSets demonstrates a more progressive learning curve that ultimately reaches a lower error.

5.2.4 Predictions compared to ground truth GED

The scatter plots in Figures 7 and 8 compare predicted GED values against the ground truth on the test set.

In both cases, predictions are strongly aligned with the identity line, indicating that both models successfully approximate the edit distance.

DeepSets exhibits a tight concentration of points around the diagonal across the full range of GED values, suggesting low prediction variance and good fit. SinkhornGED also demonstrates strong correlation, but with slightly increased dispersions.

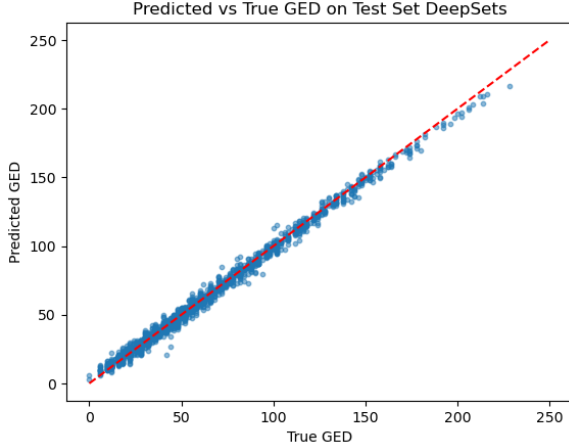


Figure 7: Scatterplot of predicted and ground truth GED values for DeepSets

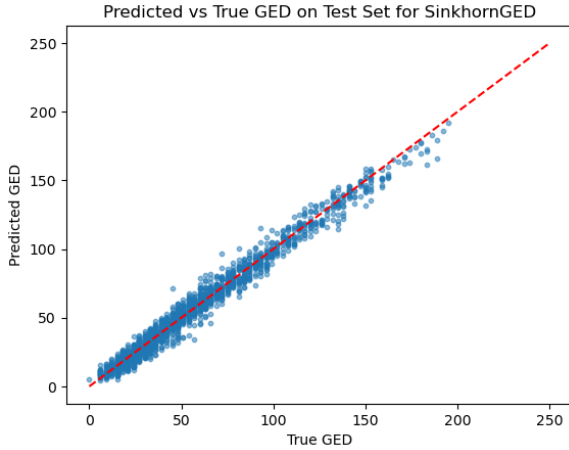


Figure 8: Scatterplot of predicted and ground truth GED values for Sinkhorn

Moreover, SinkhornGED shows a mild tendency to underestimate larger GED values.

Overall, the results on the SAMPL dataset show that the relative performance of the models depends slightly on how the GED is defined. In the simplified setting, Sinkhorn-GED has a clear advantage. This is expected, since the model explicitly computes a soft assignment matrix and is therefore well aligned with the bipartite matching formulation of the target.

When moving to the more complex costs, DeepSets performs substantially better, suggesting that learning a global representation can be more effective.

From a practical perspective, DeepSets is also computationally lighter and trains significantly faster.

6 Conclusion

In this work, we conducted a comprehensive comparison between permutation-invariant DeepSets models and Sinkhorn-based optimal transport architectures for learning similarity between unordered structured objects. Across both synthetic point cloud data and molecular graph GED prediction tasks, we observed that both approaches are capable of accurately approximating combinatorial matching-based similarity measures, achieving high correlation with ground-truth distances.

On synthetic datasets with Euclidean and Hausdorff metrics, DeepSets and Sinkhorn performed comparably, with DeepSets showing slightly better stability and robustness to increasing feature dimensionality. Interestingly, both models benefited from larger set cardinalities, suggesting that richer set-level statistics facilitate learning. However, Sinkhorn exhibited greater sensitivity to high-dimensional inputs, likely due to the behavior of pairwise distances in high-dimensional spaces.

On the molecular GED task, the relative performance depended strongly on the cost structure. In the simplified node-matching regime, the Sinkhorn-GED model outperformed DeepSets, demonstrating the advantage of explicitly modeling bipartite assignments when the distance is dominated by matching operations. In contrast, under complex edit costs involving both nodes and edges, DeepSets significantly surpassed Sinkhorn-GED, indicating that global structural aggregation can be more effective when similarity depends less on precise correspondences and more on global discrepancies.

Overall, our findings suggest that Sinkhorn-based models are most beneficial when the target similarity closely mirrors a true assignment problem, whereas DeepSets offers a robust, computationally efficient alternative in general to computing the optimal assignments. A limitation of this work is its focus on specific synthetic and molecular domains. Future research might therefore extend this evaluation to a broader range of graph datasets and alternative similarity metrics to assess generalization. Furthermore, while our current architectures provide strong baselines, there still remains a potential for improvement.

References

- Luc Brun, Benoit Gaüzère, Guillaume Renton, Sébastien Bogleux, and Florian Yger. 2022. [A differentiable approximation for the linear sum assignment problem with edition](#). In *2022 26th International Conference on Pattern Recognition (ICPR)*, pages 3822–3828.
- Marco Cuturi. 2013. [Sinkhorn distances: Lightspeed computation of optimal transportation distances](#). *Preprint*, arXiv:1306.0895.
- Francesco Leonardi, Markus Orsi, Jean-Louis Raymond, and Kaspar Riesen. 2025. [Gedan: Learning the edit costs for graph edit distance](#). *Preprint*, arXiv:2508.03111.
- Gabriel Peyré and Marco Cuturi. 2020. [Computational optimal transport](#). *Preprint*, arXiv:1803.00567.
- Kaspar Riesen and Horst Bunke. 2009. [Approximate graph edit distance computation by means of bipartite graph matching](#). *Image and Vision Computing*, 27:950–959.
- SAMPL challenges. 2024. Sampl challenges. <https://samplchallenges.org/>.
- Britt van Leeuwen, Sandjai Bhulai, and Rob van der Mei. 2025. [Combining style and semantics for robust authorship verification](#). *Machine Learning with Applications*, 22:100732.
- Julia Wallnig, Luc Brun, Benoit Gaüzère, Sébastien Bogleux, Florian Yger, and David B. Blumenthal. 2025. A differentiable approximation of the graph edit distance. In *Structural, Syntactic, and Statistical Pattern Recognition*, pages 1–10, Cham. Springer Nature Switzerland.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. 2018. [Deep sets](#). *Preprint*, arXiv:1703.06114.